PortfolioOS — Comprehensive Technical Documentation

1. Executive Summary

PortfolioOS is a state-of-the-art, web-based Personal Operating System designed to serve as an interactive, highly dynamic personal portfolio and administrative portal. Built on a modern full-stack JavaScript/Node.js architecture, PortfolioOS simulates a complete desktop environment directly within the web browser—featuring draggable/resizable windows, an interactive dock, dynamic taskbar, custom terminal CLI, multi-theme engine, and a multi-tiered administrative control center.

Beyond visual excellence, PortfolioOS incorporates enterprise-grade security mechanisms, including **Bcrypt pin hashing**, **session token management**, **IP-based rate limiting with automatic lockout protection**, and a **Double Authentication system (PIN + Email OTP)** for sensitive administrative tasks like PIN resets, email updates, and full portfolio data wipes.

---

2. Technology Stack

2.1 Backend Stack

- **Node.js**: Asynchronous JavaScript runtime environment.
- **Express.js (`v5.2.1`)**: Fast, lightweight web framework providing RESTful API routes, security middleware, and session handlers.
- **MongoDB & Mongoose (`v9.6.2`)**: NoSQL database and Object Data Modeling (ODM) layer for schema enforcement and data persistence (`User`, `UserConfig`, `Otp` collections).
- **Bcrypt.js (`v3.0.3`)**: Password-hashing library implementing 10-round salted bcrypt hashing for PIN security.
- **Nodemailer (`v9.0.3`)**: Email delivery engine supporting both production SMTP servers and automated fallback to **Ethereal Email** for local development testing.

- **CORS (`v2.8.6`)**: Cross-Origin Resource Sharing middleware configured with dynamic domain whitelisting.
- **Dotenv (`v17.4.2`)**: Zero-dependency environment variable loader for managing database URIs, ports, and credentials.

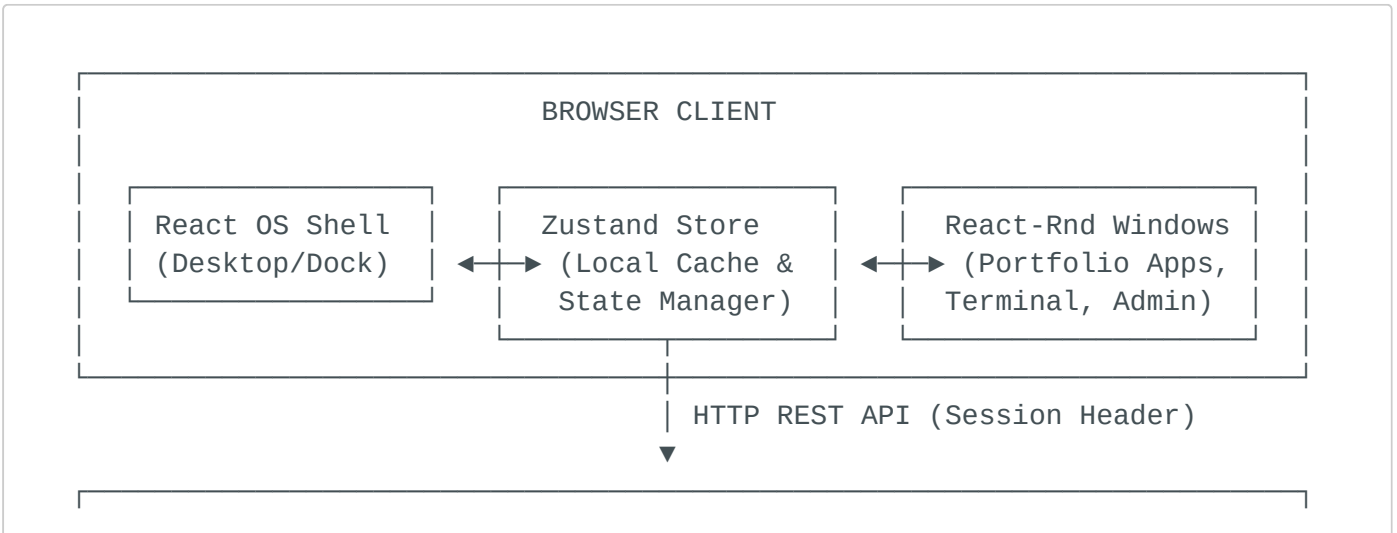
2.2 Frontend Stack

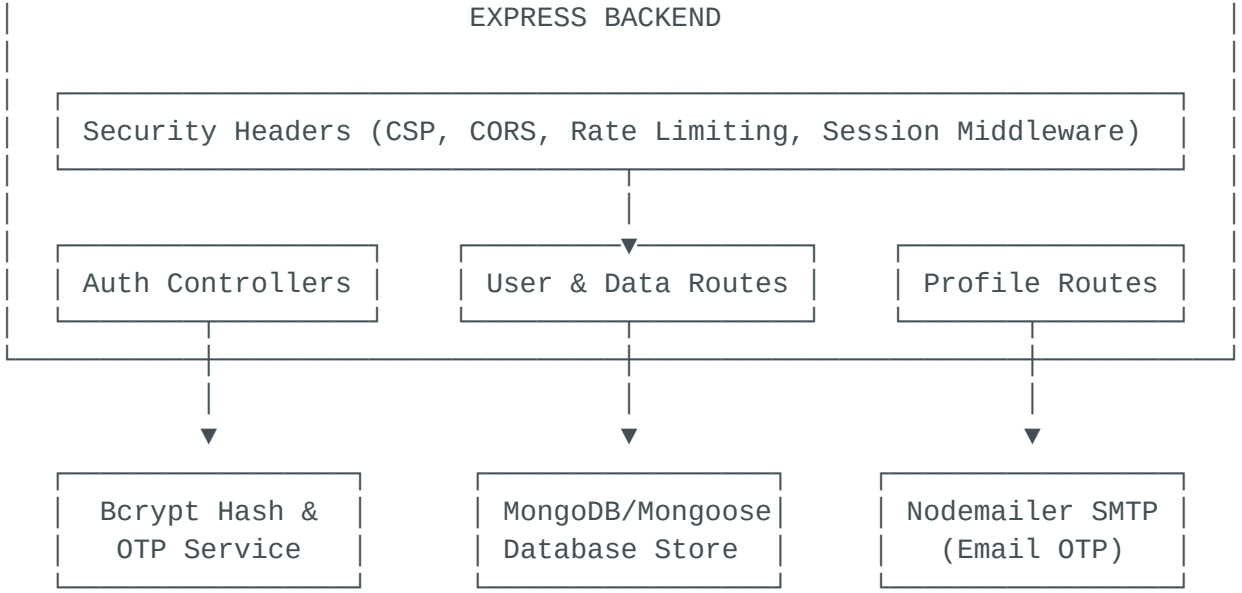
- **React (`v19.2.7`)**: Modern UI library utilizing functional components, hooks, and React 19 concurrent features.
- **Vite (`v8.1.1`)**: Next-generation frontend build tool providing Instant Hot Module Replacement (HMR) and optimized ES module bundling.
- **Tailwind CSS (`v4.3.2`)**: Utility-first CSS framework with Tailwind v4 engine integration for fast layout styling and theme variable management.
- **Zustand (`v5.0.14`)**: Lightweight, high-performance central state management store for window states, user preferences, active sessions, and data sync.
- **Framer Motion (`v12.42.2`)**: Production-ready animation library driving desktop transitions, window opening/closing animations, dock magnification, and modal effects.
- **React-Rnd (`v10.5.3`)**: Draggable and resizable container component wrapper empowering the OS window system.
- **Lucide React & React Icons (`v1.23.0` & `v5.7.0`)**: Vector icon libraries for dock items, window controls, navigation items, and system indicators.

---

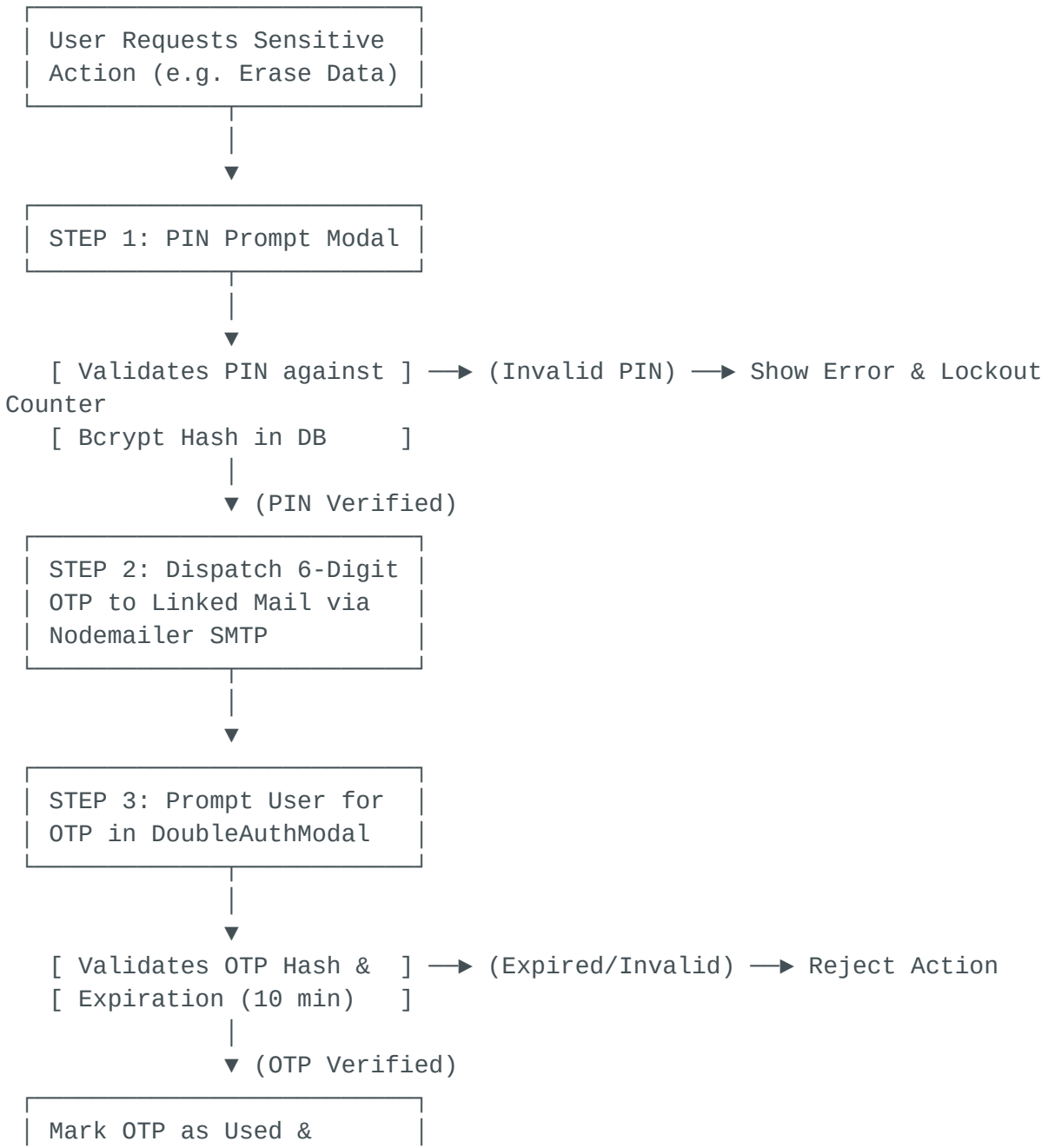
3. System Architecture & Flow Diagrams

3.1 High-Level Architecture Overview





3.2 Double Authentication Security Flowchart



```
| Execute Privileged Action|
└──────────────────────────┘
```

---

4. Backend Architecture & Functions

4.1 Server Middleware & Security Hardening (`server.js`)

- `X-Content-Type-Options: nosniff`: Prevents MIME type sniffing.
- `X-Frame-Options: DENY`: Protects against clickjacking.
- `X-XSS-Protection: 1; mode=block`: Enables browser XSS filters.
- `Content-Security-Policy (CSP)`: Enforces strict origin restrictions for script, style, and font resources.
- `rateLimitAuth(req, res, next)`: Tracks login attempts by IP. If failed attempts exceed `MAX_ATTEMPTS = 10`, locks out the IP for `LOCK_DURATION_MS = 120,000ms` (2 minutes).
- `requireSession(req, res, next)`: Validates incoming request headers (`x-session-token`) against memory session store `activeSessions`. Expires stale sessions automatically after 1 hour.

4.2 Database Models

1. **`User.js`**: Stores admin/user profiles.
    - Fields: `mailId`, `displayName`, `avatar`, `accessPinHash`, `isVerified`, `accountType`, `failedAttempts`, `lockedUntil`.
2. **`Otp.js`**: Handles single-use verification tokens.
    - Fields: `mailId`, `otpHash`, `purpose`, `expiresAt`, `isUsed`.
    - Index: Native MongoDB TTL index automatically prunes documents 5 minutes after expiration.
3. **`UserConfig`**: Stores full portfolio JSON payloads (`projects`, `skills`, `certificates`, `experience`, `education`, `settings`).

4.3 Key Backend Functions

Function Name	Description	Security / Logic
`getStoredHash()`	Fetches active admin PIN hash from MongoDB `User` or `UserConfig`.	Falls back to in-memory cache if DB is offline.
`verifyPassword(plaintext)`	Compares submitted PIN with stored salted hash using `bcrypt.compare`.	Prevents timing attacks and plaintext leakage.
`saveNewPasswordHash(plaintext)`	Hashes new PIN using 10 bcrypt rounds and updates DB records.	Atomic updates across `User` and `UserConfig` collections.
`createSession(mailId, isAdmin)`	Generates 32-byte cryptographic hex token stored in memory map.	Auto-purges expired sessions with `setTimeout`.
`pruneExpiredCertificates(data)`	Scans user certificates and removes items older than 10 minutes (addedAt).	Keeps temporary session certificates clean.
`sendOTP(to, otp, purpose)`	Formats purpose-specific HTML email and dispatches via Nodemailer.	Automatically handles Ethereal dev fallback if SMTP is absent.

5. Frontend Architecture & State Management

5.1 Central Store (`frontend/src/store/useStore.js`)

State is powered by **Zustand**, serving as the reactive bridge between UI components and backend REST endpoints:

- **Window Management State**:
  - `windows`: Array of open window objects (`{ id, title, zIndex, isMinimized, isMaximized }`).
  - `openWindow(id, title)`: Opens a new window or brings an existing window to front with incremented `zIndex`.
  - `closeWindow(id)` / `minimizeWindow(id)` / `maximizeWindow(id)`: Controls window states dynamically.
  - `focusWindow(id)`: Updates window z-index layer without unminimizing.
- **Session & Persistence**:
  - `validateStoredSession()`: Checks localStorage token `adm_session_v2` against backend endpoint `/api/validate-session` on app start.
  - `syncBackend(userObject)`: Pushes updated portfolio payload to `/api/user` with session token headers.
- **Customization Engine**:
  - `settings`: Stores global theme settings (`theme`, `fontStyle`, `fontSize`, `accentColor`, `wallpaper`, `animations`).
  - `updateSetting(key, value)`: Updates state and persists setting immediately to `localStorage` and `document.documentElement` attributes.

5.2 Component Architecture & Applications

1. **`Desktop.jsx`**: Main wallpaper grid, handles desktop shortcuts, context menus, and active window rendering.
2. **`Dock.jsx`**: Animated macOS-style dock with Framer Motion hover magnification, bouncing launch icons, and indicator dots.
3. **`MenuBar.jsx`**: Top status bar displaying system clock, Wi-Fi status, theme toggles, active app name, and quick admin settings.
4. **`Window.jsx`**: Window wrapper utilizing `react-rnd` for fluid drag/resize capabilities, OS action buttons (close, minimize, maximize), and focus handling.

5. `TerminalApp.jsx`: Full-fledged command-line interface supporting commands like `help`, `about`, `skills`, `projects`, `sudo`, `matrix`, `theme`, `clear`, and `contact`.
6. `AdminApp.jsx`: Administrative control center for managing portfolio bio, skills, experience, projects, education, certificates, and system security.
7. `AccountProfile.jsx` & `DoubleAuthModal.jsx`: Profile management interface with double-authentication triggers for PIN resets and data wipes.
8. `BentoApp.jsx` & `PortfolioApp.jsx`: Modern bento grid layout presenting skills, timeline, projects, and bio in visual cards.

---

6. Advantages & Engineering Highlights

1. **Unrivaled User Engagement**: Replaces flat, static portfolios with an immersive, memorable OS experience.
2. **Zero-Latency UI Performance**: Local state modifications in Zustand update instantly while asynchronous REST calls sync with MongoDB in the background.
3. **Enterprise Multi-Factor Security**: Double authentication (PIN + Email OTP) ensures high-privilege actions cannot be executed even if a PIN is compromised.
4. **Resilient Offline Fallback**: Features graceful degradation—if MongoDB is temporarily offline, the server falls back to memory caches without crashing.
5. **Comprehensive Customization**: Dynamic theme switcher dynamically modifies CSS root variables, providing live previewing of fonts, accent colors, and dark modes.

---

7. Security Architecture & Threat Mitigation

Security Aspect	Threat Mitigated	Implementation Detail
Bcrypt Hashing	Database Leak / Credential Exposure	10 rounds salted hashing for all PINs. Plaintext PINs are never stored.
Double Authentication	Session Hijacking / Privilege Escalation	Sensitive operations require two factors: static PIN + single-use email OTP.

Security Aspect	Threat Mitigated	Implementation Detail
IP Rate Limiting	Brute-Force Password Attacks	10 failed login attempts trigger an immediate 2-minute IP lockout.
OTP Expiration & TTL	Token Replay Attacks	OTPs expire after 10 mins and auto-delete via MongoDB TTL index.
Session Token Isolation	CSRF / Cookie Vulnerabilities	Uses custom HTTP header `x-session-token` stored in memory/localStorage.
Security Headers	XSS, Clickjacking, Sniffing	Enforces CSP, X-Frame-Options DENY, X-Content-Type-Options nosniff.